Network Intrusion Detector (Scikit-learn)

A machine learning-based **Network Intrusion Detection System (NIDS)** built with **Scikit-learn** to classify network traffic anomalies and cyber-attacks using public datasets (such as NSL-KDD or KDD Cup 99). [1, 2, 3]

Features

- **Data Preprocessing & Scaling**: Handles categorical encoding, missing value imputation, and feature scaling using `StandardScaler` or `MinMaxScaler`. [1, 2]
- **Multiple Classifiers**: Trains and evaluates supervised learning models using Scikit-learn:
  - Logistic Regression
  - K-Nearest Neighbors (KNN)
  - Support Vector Machine (SVM)
  - Decision Trees & Random Forest [1, 2, 3]
- **Evaluation Metrics**: Computes **Accuracy, Precision, Recall, F1-Score**, and **Confusion Matrices**. [1, 2]
- **Binary & Multi-class Classification**: Distinguishes normal traffic from attack categories (e.g., DoS, Probe, R2L, U2R).

```
network-intrusion-detector/
│
├── data/                # Place raw/processed datasets (e.g., NSL-KDD)
├── notebooks/           # Exploratory Data Analysis (EDA) Jupyter notebooks
├── src/
│   ├── preprocess.py    # Data cleaning, encoding, and scaling functions
│   ├── train.py         # Model training and hyperparameter tuning
│   └── evaluate.py      # Metrics calculation and confusion matrix plotting
├── requirements.txt     # Project dependencies
└── README.md            # Project documentation
```

Tech Stack

- **Language**: Python 3.8+
- **Machine Learning**: `scikit-learn`
- **Data Manipulation**: `pandas`, `numpy`

- **Visualization**: `matplotlib`, `seaborn` [1, 2, 3]

Installation & Setup

1. **Clone the repository**:

   bash

   ```
   git clone https://github.com
   cd network-intrusion-detector
   ```

   Use code with caution.
2. **Create a virtual environment (optional but recommended)**:

   bash

   ```
   python -m venv venv
   source venv/bin/activate   # On Windows: venv\Scripts\activate
   ```

   Use code with caution.
3. **Install dependencies**:

   bash

   ```
   pip install -r requirements.txt
   ```

   Use code with caution.

Usage

1. **Prepare the Dataset**:

   Download a public dataset (like NSL-KDD) and place the CSV files inside the `data/` directory.
2. **Run Preprocessing**:

   bash

   ```
   python src/preprocess.py
   ```

   Use code with caution.

3. **Train the Model**:

   bash

   ```
   python src/train.py
   ```

   Use code with caution.
4. **Evaluate Performance**:

   bash

   ```
   python src/evaluate.py
   ```

   Use code with caution.

Results Summary

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	~98.5%	~98%	~98%	~98%
K-Nearest Neighbors (KNN)	~97.2%	~97%	~97%	~97%
Logistic Regression	~92.0%	~91%	~92%	~91%